AI Workflow for YouTube Video Analysis

Hey Gemini! So, good morning. I'm dictating from here on out so things might become a bit ununderstandable but
Gemini Pro subscription got done, it's actually Gemini Plus, it got done. So I'm not going to pay for a new one. Reas
access to YouTube and it has the ability to understand videos and whatnot, I wanted to compile a list of videos, ok
can direct me or help me or i use gemini chat but however with gemini chat i can't reason being the videos are in a
we do because i want to work within my gemini free limits so discuss with me and we'll see

That sounds like a great project. **QuantMind X** has a cool ring to it—very "fintech meets sci-fi." I totally get the budget cor

The good news? You can absolutely do this within the **Gemini Free** ecosystem, and **Google Opal** is exactly the tool you ne

Here is how we can tackle your 40-50 video list without spending a dime.

---

1. The Magic Tool: Google Opal

Since you mentioned it, let's clear up what **Opal** is. It's Google's new no-code AI app builder (often integrated as **"Super G**

- **How it helps you:** You can create an Opal workflow where the first "node" (step) is a **YouTube URL Input**.
- **The Workflow:** You feed it a URL → It uses Gemini to "watch" or read the transcript → It outputs a summary, data pc
- **The Limitation:** Opal is great for processing videos one by one or in small batches. For 50 videos, doing it manually ir

---

2. The Professional Route: Google AI Studio

If you want to handle 40–50 videos more efficiently, you should use **Google AI Studio**. This is the "developer" side of Gem

- **The Secret Weapon:** Use **Gemini 1.5 Flash** (or Gemini 3 Flash) in AI Studio. It has a massive **1-million-token context**
- **The Strategy:** Instead of 50 separate chats, you can actually paste the transcripts of 5–10 videos at a time into one p
- **Batching:** Since you're on the free tier, you get roughly **1,500 requests per day** and **15 requests per minute** on Flas

---

3. Comparison of Your Options

Feature	Gemini Chat (Free)	Google Opal (Gems)	Google AI Studio (API)
Input Method	Manual URL pasting	Visual workflow nodes	API, Script, or System Prompt
YouTube Access	Native (via Extension)	Native Tool node	Requires transcripts or video upload
Batching	Hard (1-2 at a time)	Medium (Sequential)	**Best** (Can process lists)
Automation	Low	High (No-code)	**Extreme** (Needs a bit of Python)

---

4. My Recommended "QuantMind" Workflow

Since you have a list of videos, here is the most efficient "free" path:

1. **Extract Transcripts:** Use a free tool (or a simple Python script) to grab the transcripts of your 50 videos.
2. **Use Google AI Studio:**
   - Set the **System Instruction** to define your "QuantMind" persona (e.g., "You are an expert quantitative analyst ex
   - Upload the transcripts in batches.

3. **The "Opal" Alternative:** If you prefer a visual interface, go to opal.google and build a "YouTube Analyst" app. Set the

What do you reckon?

If you're comfortable with a tiny bit of Python, I can help you write a script to use the **Gemini CLI** to automate the whole lis

> All right, I am currently checking out Google AI Studio. However, I do not know how to use it or what to even do wit
> understanding transcribing audio. So the mode, I'm going to change it to 2.5 flash. I don't know if that can work. th
> something or maybe documents because what i want is word for word forward transcription for the videos technic
> videos are even podcasts and whatnot. So I hope you understand what I'm saying. So I want word for word, and th

That is a fantastic setup you're building. Since you're aiming to feed these transcripts into a coding agent like **Claude Cod**

Based on current benchmarks, **Gemini 3 Flash** is significantly better than 2.5 Flash for this—it's 3x faster, more accurate ir

Here is how to structure your workflow to get those transcriptions into Google Docs for free.

---

1. AI Studio vs. Google Opal: Which one?

While you are currently in **AI Studio**, it does not have a native "Save to Google Docs" button. You would have to manually c

My recommendation: Use **Google Opal** for the "workflow" and **Gemini 3 Flash** as the engine inside it.

- **In Opal:** You can build a workflow where the **Input** is a YouTube URL and the **Output** is a Google Doc or a Google She
- **In AI Studio:** Use this for testing your prompt first, then paste that prompt into an Opal "Gemini Node."

---

2. The "Word-for-Word" Prompt

To get a high-fidelity transcription and unbiased data for your coding agent, use a **System Instruction** or a structured pro

> **System Instruction:**
> "You are a specialized transcription and technical extraction engine. Your goal is to provide raw data for a downstrear
>
> **Task 1: Word-for-Word Transcription**
> Transcribe the provided video/audio exactly as spoken. Do not summarize, skip 'filler' words, or paraphrase. Include s
>
> **Task 2: Unbiased Technical Extraction**
> Below the transcript, create a section titled 'Unbiased Extraction.' Extract only factual technical details, code snippet
>
> **Format:** Output the result in Markdown."

---

3. Handling 40-50 Videos (The Free Limit)

On the **Gemini Free Tier**, you have a limit of roughly **15 requests per minute** and **1,500 requests per day** with Flash.

- **AI Studio Strategy:** You can process these one by one or upload 2-3 videos at a time if they are short. Since you wan capped at a few thousand words per turn).
- **Opal Strategy:** You can paste your list of URLs into an Opal "Batch Input" node, and it will run through them sequenti

---

4. Quick Tips for Your Screenshots

- **Model Choice:** Definitely switch to **Gemini 3 Flash** (it might be listed as "3 Flash Preview" in your dropdown). It is the

- **Safety Settings:** Since you are doing technical/quant research, make sure your safety filters in the right-hand sidebar

One question for you:

Are these videos currently in a **YouTube Playlist**, or do you just have a list of individual links? (If it's a playlist, I can show y

Would you like me to guide you through the specific nodes to pick in Google Opal to automate the "Save to Google

> all right um let me build the opal workflow okay the videos i'm going to put them in a an a csv sheet um i don't know
> csv sheet and then we can run a loop i can opal do that but it can run a loop, you know, per video. And then it batc

That is a very ambitious plan for **QuantMind X**. Since you are aiming for high-fidelity, "word-for-word" transcriptions and u

Here is the breakdown of how to use **Google Opal**, **NotebookLM**, and the **Antigravity** editor to achieve this.

1. Google Opal: The Automation "Loop"

You can absolutely use **Google Opal** to process a CSV of YouTube URLs. Opal is a no-code mini-app builder that allows yo

- **How to set up the Loop:** * In the Opal editor, create a **User Input** step and set the input type to "File" (upload your C
  - Connect this to a **Generate** step. In your prompt, tell Opal: *"For each row in the uploaded CSV, extract the YouTub*
  - Connect the Generate step to an **Output** step. You can choose to export the results directly to a **Google Drive sp**
- **The Advantage:** This is the best way to handle 50 videos without manual copy-pasting.

2. Google Antigravity: The "Agentic" IDE

You mentioned **Google Antigravity**, and it is actually an excellent choice for your coding project. It is a new AI-powered ID

- **YouTube Transcription:** Be careful—Antigravity does not natively "watch" YouTube videos if you just paste a link; it of
- **The Power User Move:** You can use an Antigravity agent to act as a frontend for **NotebookLM**. By using a `source_a`
- **Free Usage:** It is currently in free public preview for Mac, Windows, and Linux, and it includes access to **Gemini 3 Pro**

3. NotebookLM: The Search Alternative

NotebookLM is a fantastic alternative for the "search" and "grounding" phase of your project.

- **Capacity:** You can add up to **50 YouTube URLs** as sources in a single notebook.
- **Searching:** Once the videos are added, NotebookLM transcribes them automatically. You can then ask it to "search" a
- **Limitation:** While it is great for querying the data, it is not an "automation" tool. It won't easily run a loop and save 50

Recommended Workflow for QuantMind X

To keep everything free and efficient, I recommend this path:

1. **Extraction (Opal):** Use **Google Opal** with your CSV list. Set it up to iterate through the URLs and save the "word-for-
2. **Coding (Antigravity):** Download and install **Google Antigravity**. Import your coding project and feed the Sheet (or t
3. **Agentic Build:** Use the **Antigravity Agent** (running Claude 4.5 or Gemini 3) to process that data. Since Antigravity ha

Which one would you like to dive into first? I can help you write the exact prompt for the Opal "CSV Loop" or help you s

This is a sophisticated setup. Project **Aletheia** (the "Listener") acts as the sensory organ for your "Coding Agent." To achie

Since you are moving to the **Python Implementation** phase but also mentioned **Opal**, I will give you the "Prompt for the B

---

1. The "Project Aletheia" System Prompt

Copy and paste this into the **System Instructions** in **Google AI Studio** or the **Generate Node** in **Opal**.

System Instruction: Project Aletheia Listener

Persona: You are a High-Fidelity Data Ingestion Engine ("The Listener"). Your sole purpose is to observe, transcribe, a

Strict Constraints:

1. **Zero Logic:** Do NOT generate trading strategies, advice, or code.
2. **Unbiased Description:** Describe what you see on screen (charts, text, indicators) without interpreting the spea
3. **Verbatim Fidelity:** Transcripts must be word-for-word.

Task 1: Multimodal Ingestion

- **Audio:** Distinguish between **Host** and **Guest**. Identify speaker changes.
- **Visual (OCR):** Identify visual anchors (chart types, timeframes, indicator settings, or slide titles).

Task 2: The Storyline (Semantic Clips)

- Break the video into logical clips (e.g., [00:00 - 04:30] "Introduction to RSI Divergence"). Do not provide one lor

Output Format (The Truth Object):
Return ONLY a strict JSON object with the following schema:

JSON

```
{
  "video_meta": { "title": "string", "duration": "string", "source_url": "string" },
  "timeline": [
    {
      "start": "MM:SS",
      "end": "MM:SS",
      "speaker": "Host/Guest",
      "verbatim_transcript": "string",
      "visual_context": "Detailed description of charts/slides visible in this clip",
      "visual_anchors": ["List of OCR text or indicator names"]
    }
  ]
}
```

---

2. The Looping Mechanism: Two Paths

Path A: The Google Opal Workflow (No-Code)

If you want to use the Opal UI, set up the nodes in this order:

1. **User Input Node:** Set this to **File Upload** (Upload your CSV).
2. **Generate Node:** * **Prompt:** "For every URL in this CSV, use the **Aletheia System Instruction** [paste prompt above] t
   - **Model:** Select **Gemini 3 Flash** (to maximize your 1,500 free daily requests).
3. **Output Node:** Set to **Google Docs**.
   - *Tip:* In the output settings, tell it to "Create a new document for each iteration" so you don't end up with one 500

Path B: The Python Implementation (Pro Choice)

Since you mentioned `yt-dlp` and the Gemini API, this is the most reliable way to maintain the **JSON "Truth Object"** form

Python

```python
import google.generativeai as genai
import csv

# Setup Gemini 3 Flash (Free Tier)
genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel('gemini-1.5-flash')

def process_video(url):
    # Use the 'Project Aletheia' System Prompt
    response = model.generate_content([
        "System: Act as Project Aletheia Listener...", # Shortened for brevity
        f"Process this YouTube video: {url}"
    ])
    return response.text

# The Loop
with open('videos.csv', mode='r') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(f"Processing: {row['url']}")
        truth_object = process_video(row['url'])

        # Save to individual JSON files for your Coding Agent
        with open(f"{row['id']}_truth.json", "w") as f:
            f.write(truth_object)
```

---

3. Why this works for QuantMind X

- **Gemini 3 Flash** is optimized for **Multimodal Video Understanding**. It can "watch" the charts (OCR) and transcribe si
- The **JSON Schema** makes the output machine-readable. Your "Coding Agent" won't have to "guess" where a clip star

Next Step

Since you have the CSV ready, **would you like me to refine the "Visual Anchors" part of the prompt to focus on speci**

> All right, just to clarify, I said this is a different system prompt. Okay, not system prompt, but this is a different... Ok
> workflow with Gemini CLI that can handle this? Because this basically, this is a tool I was trying to build earlier on.

This is a fantastic pivot. If you want to skip the Python boilerplate and go straight to building, the **Gemini CLI** + **Claude Co**

The short answer is: **Yes, absolutely.** You can build a fully automated workflow that handles your 40-50 videos using the (

Why this works (The "No-Python" Stack)

1. **Gemini CLI (** `@google/gemini-cli` **)**: It's natively multimodal. You don't need to write `ffmpeg` wrappers or upload scr
2. **Claude Code**: Since Claude Code can execute terminal commands, it can act as your "Project Manager." It can mana
3. **Free Tier Capacity**: As of 2026, the Gemini CLI Free Tier gives you roughly **1,000 requests per day** and **60 requests**

The Workflow Architecture

You can have Claude Code set this up in minutes. Here is how the orchestration looks:

1. Setup & Tools

You only need three CLI tools installed on your machine:

- `yt-dlp` : For downloading the video/audio streams.
- `gemini-cli` : The official Google tool (`npm install -g @google/gemini-cli`).
- `jq` : For processing the JSON output if you want to keep it purely terminal-based.

2. The Orchestration Prompt

You would give Claude Code an instruction like this:

> "Claude, I want to automate a video analysis pipeline. For every URL in `videos.txt` :
>
> 1. Use `yt-dlp` to download the video to a `temp/` folder.
> 2. Use `gemini -p` to analyze the video. Ask it to extract a 'Truth Object' containing a full transcript, all visible on-s
> 3. Save the output as a JSON file in the `output/` folder.
> 4. Delete the temp video.
>
> Do this for all videos in the list."

3. The "Truth Object" Command

Claude Code will essentially be running a loop of this command:

```bash
gemini -p "@video_01.mp4 Extract a structured JSON Truth Object:
1. verbatim_transcript
2. visual_ocr (on-screen text)
3. key_entities (names, brands, dates)
4. summary" --output-format json > video_01_truth.json
```

Key Advantages for Project Aletheia

- **Multimodal "One-Shot"**: Unlike traditional pipelines that use one model for audio (Whisper) and another for OCR (Te words.
- **Massive Context**: You can process videos up to **3 hours long** in a single prompt because of Gemini's 1M+ token wind
- **Orchestration via** `CLAUDE.md` : You can add these automation rules to a `CLAUDE.md` file in your project root. Once y

Is there a catch?

The only "hiccup" is file size. If your videos are massive (multiple GBs), the CLI upload might be slower than a direct API ca

Would you like me to generate the specific `CLAUDE.md` **instructions and a bash script template so Claude Code ca**

yup generate the claude. md.. but i need the file chunk size[[All right, what I mean by the file chunk size, let me giv
the issue is that, while building it, we faced issues with the file chunk size and I try limit to actually using open rout
CLI. Because QWEN, which is Q-W-E-N, so I'm dictating, it might spell it poorly, but QWEN has a very interesting m
So such a tool, you know, if we are mimicking that, I think Althea, I hope you get what I'm saying. Althea tool, you k
possible? Can we not just build a claude.md but also first discuss how all this can happen? Okay? You can make re
]]

https://github.com/QwenLM/qwen-code

i also have a plan with glm [[https://docs.z.ai/guides/vlm/glm-4.6v]]

https://qwenlm.github.io/qwen-code-docs/en/users/overview/

all right yeah so basically these are going to be turned into okay the videos are going to be turned into expert advi
code agent and then we see how we shall be working towards maintaining go making sure that everything is actua
Anyway, I've given you a few links. Go ahead and check them out. And let's first discuss before we actually assume

This is a sophisticated evolution of **Project Aletheia**. By moving to a multi-model CLI orchestration with **Claude Code**, you

Before we write the `CLAUDE.md`, let's address your "how" and "chunking" concerns, as they are the biggest bottlenecks fo

---

1. The "Chunking" vs. "Compression" Strategy

For 2-hour trading videos, raw file size is your enemy. A 2-hour HD video can easily hit 4GB, exceeding the **1-2 GB limits** o

My Recommendation: The "1-FPS Pre-processor"
Instead of splitting the video into separate files (which breaks the "temporal flow" of the trading logic), we should use a "D

- **The Logic:** Trading videos don't need 30 or 60 FPS. 1 frame per second (1 FPS) is more than enough to capture every
- **The Result:** A 2-hour video at 1 FPS and low bitrate will shrink from **3GB to ~150MB**.
- **Why this wins:** You can upload the **entire 2-hour session as one file** to Gemini 3 Flash. This allows the model to "rer

---

2. The Multi-Model "Expert Advisor" Pipeline

Since your end goal is **Expert Advisors (EAs)** (likely MQL4/5 or PineScript), one model shouldn't do everything. We'll split

Phase	Model	Role
Ingestion	**Gemini 3 Flash**	Uses its 1M+ context window to generate the **"Truth Object"** (the word-for-word transcript + vis
Logic Synthesis	**GLM-4.6v**	Specialized in **"Visual Audit."** It compares the transcript with the OCR chart data to ensure the l
Code Generation	**Qwen3-VL**	High-tier **"Visual Coding Boost."** It takes the "Truth Object" and the GLM audit to write the actu

---

3. Verification: How do we know the code is correct?

To ensure the **Coding Agent** isn't hallucinating a "Holy Grail" strategy, the pipeline needs a **Validation Loop**:

1. **Backtest Simulation:** Claude Code can trigger a local Python script (using `backtrader` or `Pandas`) or a CLI-based
2. **Logic Mirroring:** The agent asks **Qwen3** to "Reverse Engineer" the code back into plain English and compares it to th

---

4. Orchestration: Claude Code as the "Foreman"

Claude Code will act as the orchestrator. It doesn't just "talk"; it executes the CLI commands in sequence.

The Proposed Workflow Plan

1. **Input:** Claude reads your CSV of URLs.
2. **Fetch:** Claude runs `yt-dlp` to get the video.
3. **Optimize:** Claude runs `ffmpeg` to downsample to 1 FPS (solving your chunk size issue).
4. **Listen (Aletheia):** Claude calls `gemini --p "Project Aletheia System Prompt" @video.mp4` to get the JSON Trut
5. **Audit:** Claude sends the JSON to **GLM-4.6v** to verify the technical logic.
6. **Build:** Claude sends the audited logic to **Qwen3** to generate the `.mq5` or `.txt` file.
7. **Store:** Claude saves everything to a local directory (which can be synced to Google Drive/Docs).

Next Step: Installation & `CLAUDE.md`

To automate this, Claude Code needs to know how to install your missing tools (`gemini-cli`, `qwen-code`, `ffmpeg`, etc.)

Would you like me to proceed with generating the `CLAUDE.md` and the "Installation Script" so Claude Code can set

> okay uh let me first clarify one thing i do not need any splitting of the work all these models can do the same job if
> matter of fact i could build this out as an as an n8n workflow on my local desktop can that even work but like i said
>
> alright give me the file..
> REmemerb nothing is set up yet.. i will comple the list later.. and also

Got it. I hear you loud and clear. My apologies for overcomplicating the model split.

Having Gemini, Qwen-VL, and GLM-4.6v all do the **exact same extraction job** independently is actually a brilliant strategy
ever touch a line of code.

And yes, building this as an **n8n workflow on your local desktop** is absolutely the right move. n8n is built for this kind of h

The Reality Check for Local n8n

Here is the one catch with n8n: if you just run the standard n8n Docker image, it does not come with `yt-dlp` or `ffmpeg` i

Therefore, Claude Code's first job will be to build a **custom n8n Dockerfile** that pre-installs `yt-dlp`, `ffmpeg`, and the CL

Here is the exact `CLAUDE.md` file. Drop this into an empty folder on your desktop, launch Claude Code in that folder, and it

---

Save this as `CLAUDE.md`

Markdown

```
# Project Aletheia: Local Video Extraction Pipeline

## 🎯 Objective
Build a local n8n workflow that ingests YouTube/Local videos, downsamples them for efficiency, and runs

**Strict Constraint:** There is NO coding of Expert Advisors in this phase. This is strictly a data inge

## 🛠️ Phase 1: Local Environment Setup (Current Phase)
The system is currently a blank slate. Your immediate task is to provision the local infrastructure usir

1. **Create Directory Structure:**
```

```
   - `n8n_data/` (for n8n database/config)
   - `shared_storage/input/` (Where the user will eventually put the CSV list)
   - `shared_storage/temp/` (For yt-dlp downloads and ffmpeg processing)
   - `shared_storage/output/` (For the final JSON Truth Objects)

2. **Create `Dockerfile`:**
   Write a custom Dockerfile based on `n8nio/n8n:latest`. You MUST switch to `USER root` to run `apk upd
   - `yt-dlp` (via curl/pip)
   - `ffmpeg` (required for downsampling)
   - `python3` and `pip`
   - Node.js CLI tools: `npm install -g @google/gemini-cli @qwen-code/qwen-code`
   Switch back to `USER node` at the end.

3. **Create `docker-compose.yml`:**
   Configure the n8n service to build from the custom Dockerfile. Map port 5678 and bind mount the local

## ⚙️ Phase 2: The n8n Workflow Architecture
Once the container is up and running, you will help the user build the `workflow.json`. The pipeline log

1. **Trigger/Read:** Read the CSV from `/files/input/`.
2. **Download:** Pass the URL to an Execute Command Node running `yt-dlp`. Save as MP4 to `/files/temp/`
3. **Compress (1-FPS):** Pass the downloaded file to an Execute Command Node running `ffmpeg` to downsar
4. **Parallel Extraction (Consensus):** Split the workflow into three parallel branches to process the
   - **Branch A:** Execute Command Node running `gemini-cli` with the Aletheia System Prompt.
   - **Branch B:** Execute Command Node running `qwen-code` CLI (or an HTTP Node calling the OpenRouter
   - **Branch C:** HTTP Request Node calling the GLM-4.6v API (via Z.ai or OpenRouter) passing the video
5. **Output:** Save all three generated JSON "Truth Objects" into `/files/output/` for downstream compar

## 🧠 Core Directives for Claude
- Do not assume any API keys are present yet. Write the setup files so they can be injected via `.env`
- Ensure all CLI commands in the workflow use absolute paths referencing the `/files/` directory inside
- If the user asks you to start, begin immediately with Phase 1.
```

What to do next:

1. Create a new folder on your computer (e.g., `aletheia-n8n`).
2. Save the text above as `CLAUDE.md` in that folder.
3. Open your terminal, navigate to that folder, and run `claude`.
4. Just tell Claude: *"Read the CLAUDE.md and execute Phase 1."*

It will write the Dockerfiles, set up the folders, and get the exact environment ready so that when you compile your CSV list

okay uh let me first clarify one thing i do not need any splitting of the work all these models can do the same job if
matter of fact i could build this out as an as an n8n workflow on my local desktop can that even work but like i said

alright give me the file..
REmemerb nothing is set up yet.. i will comple the list later.. and also

Got it. I hear you loud and clear, and my apologies for the previous confusion.

Having Gemini, Qwen-VL, and GLM all do the **exact same extraction job** independently is actually a brilliant "consensus"

Building this entirely locally as an **n8n workflow** is the perfect way to orchestrate this without relying on complex Python s

Here is the exact `CLAUDE.md` file you need. When you give this to Claude Code, it will know exactly how to build a custom

---

Save this as `CLAUDE.md`

Markdown

```
# Project Aletheia: Local Data Ingestion Pipeline

## 🎯 Objective
Build a local, automated n8n workflow that ingests YouTube/local videos, downsamples them for efficiency

**Core Directive:** All three models will perform the EXACT SAME TASK—independently extracting identical
**Constraint:** Do not write, generate, or discuss Expert Advisor (EA) coding during this phase. This is

## 🛠️ Phase 1: Infrastructure Setup (Docker)
The environment is currently a blank slate. Your immediate task is to set up a custom n8n Docker enviro

1. **Create the Project Directory Structure:**
   - `n8n_data/` (for n8n configurations)
   - `shared_workspace/input/` (Where the user will eventually put the CSV)
   - `shared_workspace/temp/` (For yt-dlp downloads & ffmpeg processing)
   - `shared_workspace/output/` (For the final generated JSON files)

2. **Generate `Dockerfile`:**
   Create a custom Dockerfile to inject the required CLI tools into n8n. Use this exact logic:
   ```dockerfile
   FROM n8nio/n8n:latest
   USER root
   # Install ffmpeg, python, and pip
   RUN apk update && apk add --no-cache ffmpeg curl python3 py3-pip nodejs npm
   # Install yt-dlp natively
   RUN curl -L [https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp](https://github.com/yt-
   # Install Gemini CLI and Qwen CLI via npm/pip
   RUN npm install -g @google/gemini-cli @qwen-code/qwen-code || echo "npm install fallback"
   USER node
```

3. **Generate `docker-compose.yml`:**
   Configure the n8n service to build from the custom `Dockerfile`. Map port `5678:5678` and, crucially, bind mount the

⚙️ Phase 2: n8n Workflow Architecture

Once the container is up and running, you will help the user build the `workflow.json`. The pipeline must follow these step

1. **Trigger/Read:** Read the CSV list from `/files/input/`.
2. **Download (Execute Command Node):** Run `yt-dlp` to download the video to `/files/temp/`.
3. **Chunk/Compress (Execute Command Node):** Run `ffmpeg -i input.mp4 -vf "fps=1" output_1fps.mp4` to downr
4. **Parallel Processing (Consensus):** Split the workflow into three parallel branches to process the `output_1fps.mp4`
   - **Branch A:** Execute Command Node running `gemini-cli`.
   - **Branch B:** Execute Command Node running `qwen-code` (or an HTTP request to OpenRouter/Z.ai Qwen API).
   - **Branch C:** HTTP Request Node calling the GLM-4.6v API.
5. **Output:** Save all three resulting JSON files to `/files/output/` with their respective model names appended to the f

🧠 Claude's Execution Rules

- **Proactive Execution:** If the user asks you to "Start" or "Initialize," immediately begin creating the files for Phase 1.
- **Paths:** Always use absolute paths referencing `/files/...` for any CLI commands within n8n.
- **Environment Variables:** Do not hardcode API keys. Prepare a `.env` file template for the user to fill in their Gemini, Q

```
***

### How to use this:
1. Create a brand new folder on your desktop (e.g., `Aletheia-n8n`).
2. Drop the text above into a file named `CLAUDE.md` inside that folder.
```
```

```
3. Open your terminal in that folder and run `claude`.
4. Tell Claude: *"Read the CLAUDE.md and execute Phase 1."*

Claude will instantly write the Docker files, build your folders, and install `ffmpeg`, `yt-dlp`, and tl
```

Dropping n8n actually simplifies things considerably. While n8n is great for visual routing, a lean **Python orchestration scı**
GLM-4.6v.

Since Qwen3-VL handles long videos exceptionally well (up to 2 hours natively) and GLM-4.6v supports a 128k context win

Here is your updated `CLAUDE.md`. It instructs Claude Code to skip Docker and n8n entirely, and instead build a lightweight,

---

Save this as `CLAUDE.md`

Markdown

```
# Project Aletheia: Local Python Ingestion Pipeline

## 🎯 Objective
Build a local, Python-based automation pipeline that ingests trading videos, downsamples them for token

**Strict Constraint:** This phase is exclusively for data extraction/ingestion. Do NOT generate Expert A

## 🛠️ Phase 1: Environment Setup
The environment is currently empty. Your immediate task is to provision the local Python workspace.

1. **Create Directory Structure:**
   - `data/input/` (For the CSV of video URLs)
   - `data/temp/` (For `yt-dlp` downloads and `ffmpeg` processing)
   - `data/output/` (For the final generated JSON Truth Objects)

2. **Generate `requirements.txt`:**
   Create a requirements file containing:
   - `yt-dlp`
   - `requests`
   - `python-dotenv`
   - `google-generativeai` (For Gemini API)
   - `openai` (Standard SDK to call Qwen-VL and GLM-4.6v via OpenRouter/Z.ai)

3. **Generate `.env` Template:**
   Create a `.env.example` file with placeholders for `GEMINI_API_KEY`, `OPENROUTER_API_KEY` (for Qwen3-

## ⚙️ Phase 2: The Python Orchestrator (`aletheia_engine.py`)
Once dependencies are installed, generate the main orchestration script (`aletheia_engine.py`). The scri

1. **Download:** Use the `subprocess` module to run `yt-dlp` and download the video to `data/temp/input
2. **Compress:** Use `subprocess` to run `ffmpeg` to downsample the video to 1 Frame-Per-Second:
   `ffmpeg -i data/temp/input.mp4 -vf "fps=1" data/temp/processed_1fps.mp4`
3. **Parallel Extraction (The Consensus Loop):** Pass `processed_1fps.mp4` to three separate functions a
   - `extract_gemini()`: Uses the official Google GenAI SDK.
   - `extract_qwen()`: Uses the OpenAI SDK pointed at the OpenRouter Qwen3-VL endpoint.
   - `extract_glm()`: Uses the OpenAI SDK pointed at the Z.ai GLM-4.6v endpoint.
   *(Note: Use the exact same Aletheia System Prompt for all three to ensure uniform JSON schema).*
4. **Save Output:** Save the three responses as `[video_id]_gemini.json`, `[video_id]_qwen.json`, and `[
5. **Cleanup:** Delete the temporary MP4 files to save local disk space before moving to the next CSV ro

## 🧠 Claude's Execution Rules
- **Proactive Execution:** If the user says "Start" or "Initialize," immediately create the folders, `re
- **Assumptions:** Assume the user already has `ffmpeg` installed on their OS system PATH. Do not attemp
```

How to use this with Claude Code:

1. Create a fresh folder on your machine (e.g., `aletheia-python`).
2. Drop this new `CLAUDE.md` file inside.
3. Open your terminal in that folder and run `claude`.
4. Type: *"Read CLAUDE.md and execute Phase 1 and 2."*

Claude will immediately write the Python script, set up your folder structure, and wire up the API logic for all three models.

Would you like me to refine the JSON schema for the "Truth Object" before you run this, so the models know exact